

ChatGPT

Novo chat

Biblioteca

Agendados

Plugins

Codex

Mais

Fixada

Automacao residencial

Inteligencia Artificial

Trilha Profissional AI

Resumo dos cinco skills

Projetos

Viagem

Chats

Como arar Docker Sandboxes Do...

Otavio Lima Plus

Compartilhar

qual a diferenca para o desenvolvimento vibe coding utilizar a estrategia de virtualizar a execucao dentro do docker (via docker desktop tradicional - https://www.docker.com/products/docker-desktop/?at_exp=DO106.A) ou usar o novo produto do docker chamado docker sandbox (https://www.docker.com/products/docker-sandboxes/?at_exp=DO106.A), sendo que o link abaixo parece falar um pouco sobre isso (https://www.docker.com/blog/comparing-sandboxing-approaches-ai-agents/?at_exp=DO105.B-DO106.A)

Pensou por 9s

Sim. O artigo que vocẽ indicou trata exatamente do ponto central: **Docker Desktop tradicional e Docker Sandboxes resolvem problemas diferentes**, apesar de ambos usarem virtualizaçaõ/containers por baixo.

Para um fluxo de **vibe coding com agentes como OpenCode**, eu não trataria Docker Sandboxes como “um Docker Desktop melhor”. Eu trataria como uma **camada de execução específica para agentes autônomos**, projetada para permitir que o agente tenha muito mais liberdade sem dar essa mesma liberdade sobre sua máquina.

A diferença arquitetural mais importante

Com Docker Desktop tradicional, o padrão costuma ser algo assim:

Windows

VS Code

OpenCode

agente executa comandos

Docker Desktop

VM WSL2

container frontend

container backend

container banco

O **OpenCode** normalmente continua rodando no host. Quando ele executa:

Bash

rm -rf ...
npm install
git reset --hard
docker compose down
docker system prune

essas ações podem atingir diretamente seu workspace ou, dependendo da configuração, o Docker daemon que você usa para outros projetos.

O Docker Desktop fornece isolamento **para sua aplicação**, não necessariamente para o **agente que está controlando sua máquina**.

No Docker Sandbox, a arquitetura muda:

Windows

VS Code

Docker Sandbox

microVM exclusiva do projeto/agente

OpenCode

linux próprio

Docker Engine próprio

npm / python / gcc / etc.

containers da aplicação

Comparação direta para vibe coding

Característica	Docker Desktop tradicional	Docker Sandboxes
Objetivo principal	Executar aplicações em containers	Executar agentes de IA com segurança
Onde roda o agente	Normalmente no host	Dentro da microVM
Isolamento da aplicação	Bom	Bom

Kernel dedicado por agente	Não	Sim
Docker daemon dedicado	Normalmente compartilhado	Sim
<code>docker compose</code> pelo agente	Usa seu Docker	Usa Docker privado
<code>sudo</code> pelo agente	Perigoso no host	Permitido dentro da sandbox
Instalar pacotes	Pode alterar host/container	Pode alterar livremente a sandbox
Credenciais	Geralmente env/files	Proxy seguro de credenciais
Rede	Normal Docker networking	Policy própria, deny-by-default
Agente destruir ambiente	Pode afetar host/projetos	Normalmente limitado à sandbox
Vários agentes simultâneos	Compartilham infraestrutura	Podem ter sandboxes separadas
Descartabilidade	Containers são descartáveis	Ambiente inteiro é descartável
OpenCode	Manual/configurado por você	Suporte oficial
Docker Desktop obrigatório	Sim, nesse cenário	Não

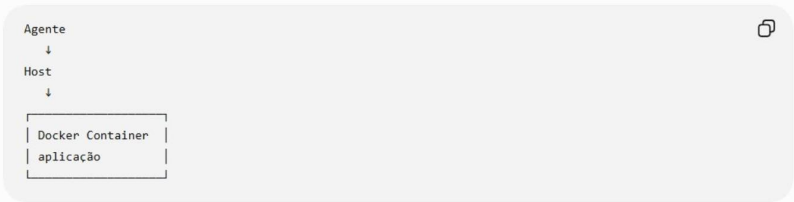
A Docker inclusive lista explicitamente **OpenCode**, **Claude Code**, **Copilot CLI**, **Codex**, **Gemini CLI** e **Kiro** entre os agentes suportados.  Docker +1

O ponto que considero mais relevante para você

Existe uma distinção conceitual importante:

Docker Desktop

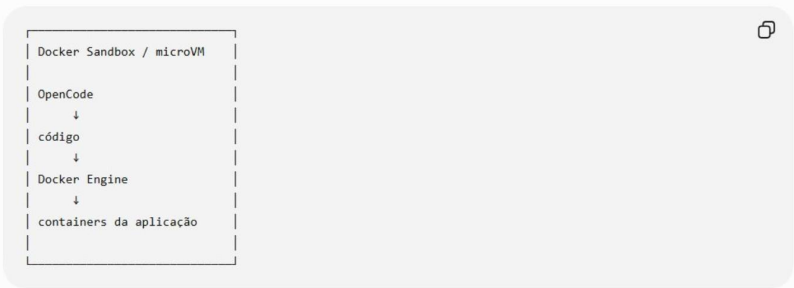
Você coloca o **software produzido pelo agente** dentro de uma caixa.



Mas o agente continua fora da caixa.

Docker Sandbox

Você coloca o **próprio agente dentro da caixa**.



Essa é uma diferença muito mais profunda do que parece.

Para desenvolvimento tradicional, isso talvez seja excesso.

Para **vibe coding/agentes**, passa a fazer bastante sentido.

Um exemplo prático

Imagine que você diga ao OpenCode:

Configure todo o projeto, instale as dependências, suba PostgreSQL, rode migrations, execute testes e corrija os erros até tudo funcionar.

No Docker Desktop tradicional, o agente pode executar:

```
</> Bash
npm install

docker compose up -d

docker exec ...

docker compose down
```

```
docker volume rm ...

git clean -fd

rm -rf node_modules
```

Você precisa confiar bastante nas decisões tomadas pelo agente.

Agora imagine que ele erre um path:

```
</> Bash

rm -rf ../../alguma-pasta
```

Ou:

```
</> Bash

docker system prune -a
```

O impacto potencial cresce muito.

O próprio artigo da Docker aponta esse problema: agentes são **não determinísticos** e podem executar operações destrutivas ou sofrer prompt injection. Containers tradicionais compartilham kernel e, quando o agente precisa executar Docker dentro de Docker, frequentemente aparecem configurações privilegiadas que reduzem significativamente o isolamento. [Docker +1](#)

No Sandbox, você pode dar muito mais autonomia

Por exemplo:

```
</> Bash

sbx run opencode ~/TESTE
```

O Docker já possui integração específica para OpenCode. [Docker Docume... +1](#)

Dentro daquele ambiente o OpenCode pode:

```
apt install
npm install
pip install
docker build
docker compose up
docker run
alterar configuração
criar banco
apagar containers
```

porque o Docker Engine que ele vê é o **Docker Engine daquela sandbox**.

Não é o Docker Engine global da sua máquina. [Docker Docume... +1](#)

Isso é particularmente interessante para agentes em modo:

```
auto approve
YOLO
dangerously-skip-permissions
```

A própria Docker apresenta Sandboxes exatamente para esse cenário. [Docker +1](#)

Mas existe uma pegadinha importante

Não interprete:

“Sandbox = agente não consegue mexer nos meus arquivos.”

Isso **não é verdade por padrão**.

Atualmente, o Docker Sandbox usa por padrão **direct mount do workspace**.

Ou seja:

```
C:\Projetos\TESTE
|
| filesystem passthrough
|
v
Docker Sandbox
|
└─ OpenCode
```

Se o OpenCode modificar:

```
src/app.ts
```

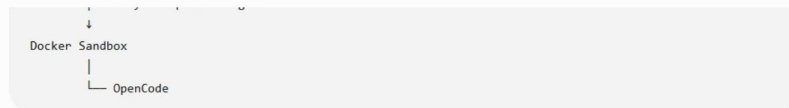
a alteração aparece imediatamente no seu host.

- criar arquivos;
- modificar arquivos;
- apagar arquivos;
- modificar `.git`;
- alterar scripts;
- alterar Git hooks.

A própria documentação deixa isso explícito. [Docker Docume... +1](#)

Isso é proposital: você quer que o código produzido pelo agente apareça no VS Code.

Existe uma modalidade ainda mais segura: `--clone`



Se o OpenCode modificar:

`src/app.ts`



a alteração aparece imediatamente no seu host.

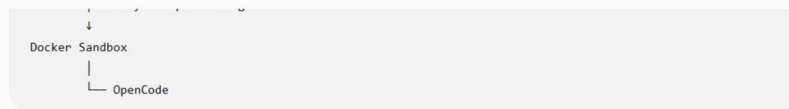
E ele pode:

- criar arquivos;
- modificar arquivos;
- apagar arquivos;
- modificar `.git`;
- alterar scripts;
- alterar Git hooks.

A própria documentação deixa isso explícito. [Docker Docume... +1](#)

Isso é proposital: você quer que o código produzido pelo agente apareça no VS Code.

Existe uma modalidade ainda mais segura: `--clone`



Se o OpenCode modificar:

`src/app.ts`



a alteração aparece imediatamente no seu host.

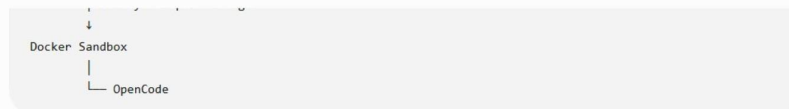
E ele pode:

- criar arquivos;
- modificar arquivos;
- apagar arquivos;
- modificar `.git`;
- alterar scripts;
- alterar Git hooks.

A própria documentação deixa isso explícito. [Docker Docume... +1](#)

Isso é proposital: você quer que o código produzido pelo agente apareça no VS Code.

Existe uma modalidade ainda mais segura: `--clone`



Se o OpenCode modificar:

`src/app.ts`



a alteração aparece imediatamente no seu host.

E ele pode:

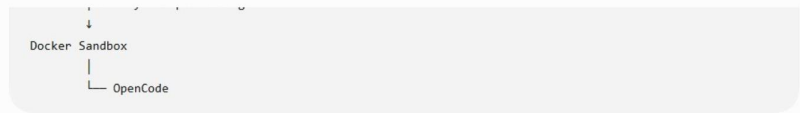
- criar arquivos;
- modificar arquivos;
- apagar arquivos;

- alterar scripts;
- alterar Git hooks.

A própria documentação deixa isso explícito. [Docker Docume... +1](#)

Isso é proposital: você quer que o código produzido pelo agente apareça no VS Code.

Existe uma modalidade ainda mais segura: `--clone`



Se o OpenCode modificar:

src/app.ts



a alteração aparece imediatamente no seu host.

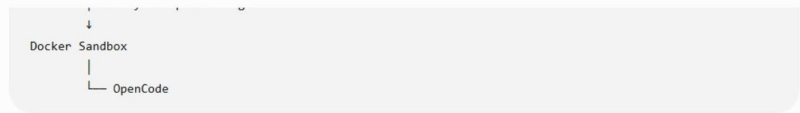
E ele pode:

- criar arquivos;
- modificar arquivos;
- apagar arquivos;
- modificar `.git`;
- alterar scripts;
- alterar Git hooks.

A própria documentação deixa isso explícito. [Docker Docume... +1](#)

Isso é proposital: você quer que o código produzido pelo agente apareça no VS Code.

Existe uma modalidade ainda mais segura: `--clone`



Se o OpenCode modificar:

src/app.ts



a alteração aparece imediatamente no seu host.

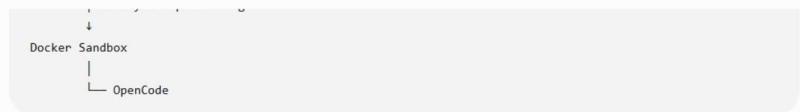
E ele pode:

- criar arquivos;
- modificar arquivos;
- apagar arquivos;
- modificar `.git`;
- alterar scripts;
- alterar Git hooks.

A própria documentação deixa isso explícito. [Docker Docume... +1](#)

Isso é proposital: você quer que o código produzido pelo agente apareça no VS Code.

Existe uma modalidade ainda mais segura: `--clone`



Se o OpenCode modificar:

src/app.ts



a alteração aparece imediatamente no seu host.

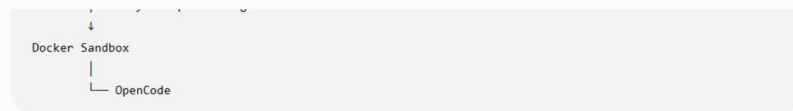
E ele pode:

- criar arquivos;
- modificar arquivos;
- apagar arquivos;
- modificar `.git`;
- alterar scripts;
- alterar Git hooks.

A própria documentação deixa isso explícito. [Docker Docume... +1](#)

Isso é proposital: você quer que o código produzido pelo agente apareça no VS Code.

Existe uma modalidade ainda mais segura: `--clone`



Se o OpenCode modificar:

`src/app.ts`



a alteração aparece imediatamente no seu host.

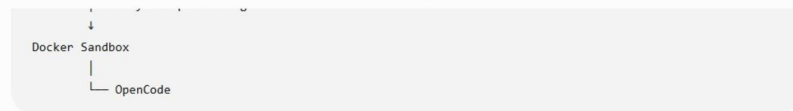
E ele pode:

- criar arquivos;
- modificar arquivos;
- apagar arquivos;
- modificar `.git`;
- alterar scripts;
- alterar Git hooks.

A própria documentação deixa isso explícito. [Docker Documente... +1](#)

Isso é proposital: você quer que o código produzido pelo agente apareça no VS Code.

Existe uma modalidade ainda mais segura: `--clone`



Se o OpenCode modificar:

`src/app.ts`



a alteração aparece imediatamente no seu host.

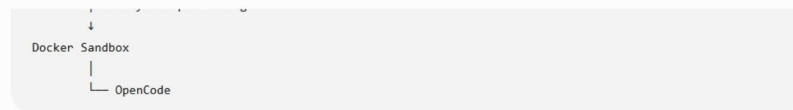
E ele pode:

- criar arquivos;
- modificar arquivos;
- apagar arquivos;
- modificar `.git`;
- alterar scripts;
- alterar Git hooks.

A própria documentação deixa isso explícito. [Docker Documente... +1](#)

Isso é proposital: você quer que o código produzido pelo agente apareça no VS Code.

Existe uma modalidade ainda mais segura: `--clone`



Se o OpenCode modificar:

`src/app.ts`



a alteração aparece imediatamente no seu host.

E ele pode:

- criar arquivos;
- modificar arquivos;
- apagar arquivos;
- modificar `.git`;
- alterar scripts;
- alterar Git hooks.

A própria documentação deixa isso explícito. [Docker Documente... +1](#)

Isso é proposital: você quer que o código produzido pelo agente apareça no VS Code.

Existe uma modalidade ainda mais segura: `--clone`



Se o OpenCode modificar:

src/app.ts



a alteração aparece imediatamente no seu host.

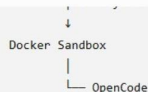
E ele pode:

- criar arquivos;
- modificar arquivos;
- apagar arquivos;
- modificar `.git`;
- alterar scripts;
- alterar Git hooks.

A própria documentação deixa isso explícito. [Docker Docume... +1](#)

Isso é proposital: você quer que o código produzido pelo agente apareça no VS Code.

Existe uma modalidade ainda mais segura: `--clone`



Se o OpenCode modificar:

src/app.ts



a alteração aparece imediatamente no seu host.

E ele pode:

- criar arquivos;
- modificar arquivos;
- apagar arquivos;
- modificar `.git`;
- alterar scripts;
- alterar Git hooks.

A própria documentação deixa isso explícito. [Docker Docume... +1](#)

Isso é proposital: você quer que o código produzido pelo agente apareça no VS Code.

Existe uma modalidade ainda mais segura: `--clone`



Se o OpenCode modificar:

src/app.ts



a alteração aparece imediatamente no seu host.

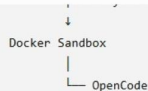
E ele pode:

- criar arquivos;
- modificar arquivos;
- apagar arquivos;
- modificar `.git`;
- alterar scripts;
- alterar Git hooks.

A própria documentação deixa isso explícito. [Docker Docume... +1](#)

Isso é proposital: você quer que o código produzido pelo agente apareça no VS Code.

Existe uma modalidade ainda mais segura: `--clone`



Se o OpenCode modificar:

src/app.ts



a alteração aparece imediatamente no seu host.

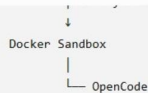
E ele pode:

- criar arquivos;
- modificar arquivos;
- apagar arquivos;
- modificar `.git`;
- alterar scripts;
- alterar Git hooks.

A própria documentação deixa isso explícito. [Docker Documentação](#) +1

Isso é proposital: você quer que o código produzido pelo agente apareça no VS Code.

Existe uma modalidade ainda mais segura: `--clone`



Se o OpenCode modificar:

src/app.ts



a alteração aparece imediatamente no seu host.

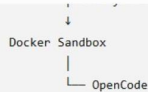
E ele pode:

- criar arquivos;
- modificar arquivos;
- apagar arquivos;
- modificar `.git`;
- alterar scripts;
- alterar Git hooks.

A própria documentação deixa isso explícito. [Docker Documentação](#) +1

Isso é proposital: você quer que o código produzido pelo agente apareça no VS Code.

Existe uma modalidade ainda mais segura: `--clone`



Se o OpenCode modificar:

src/app.ts



a alteração aparece imediatamente no seu host.

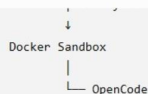
E ele pode:

- criar arquivos;
- modificar arquivos;
- apagar arquivos;
- modificar `.git`;
- alterar scripts;
- alterar Git hooks.

A própria documentação deixa isso explícito. [Docker Documentação](#) +1

Isso é proposital: você quer que o código produzido pelo agente apareça no VS Code.

Existe uma modalidade ainda mais segura: `--clone`



Se o OpenCode modificar:

src/app.ts



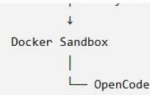
E ele pode:

- criar arquivos;
- modificar arquivos;
- apagar arquivos;
- modificar `.git`;
- alterar scripts;
- alterar Git hooks.

A própria documentação deixa isso explícito. [Docker Docume... +1](#)

Isso é proposital: você quer que o código produzido pelo agente apareça no VS Code.

Existe uma modalidade ainda mais segura: `--clone`



Se o OpenCode modificar:

`src/app.ts`



a alteração aparece imediatamente no seu host.

E ele pode:

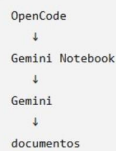
- criar arquivos;
- modificar arquivos;
- apagar arquivos;
- modificar `.git`;
- alterar scripts;
- alterar Git hooks.

A própria documentação deixa isso explícito. [Docker Docume... +1](#)

Isso é proposital: você quer que o código produzido pelo agente apareça no VS Code.

Existe uma modalidade ainda mais segura: `--clone`

isso evita um problema arquitetural sério.



que introduziria mais uma LLM no meio do caminho e aumentaria:

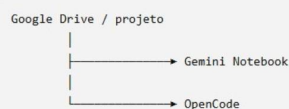
- latência;
- custo;
- perda de contexto;
- risco de interpretação;
- dependência de uma interface não documentada.

Mas é possível fazer algo mais integrado

Sim. Existem três níveis.

1. Integração simples — arquivos compartilhados

É o que eu mais recomendo inicialmente.



O Gemini Notebook aceita fontes e arquivos e trabalha estritamente sobre essas fontes em suas respostas.

[Ajuda Google +1](#)

Você mantém, por exemplo:

`docs/
architecture/
decisions/
research/
requirements/`



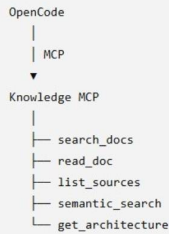
Vantagens

Pouca complexidade, nenhuma automação frágil e você mantém os documentos independentes das ferramentas.

2. Integração boa — MCP

Aqui começa a ficar realmente interessante para seu stack.

OpenCode:



No `opencode.json`:

```
<> JSON
{
  "$schema": "https://opencode.ai/config.json",
  "mcp": {
    "knowledge": {
      "...": "..."
    }
  }
}
```

O OpenCode suporta exatamente esse modelo: um servidor MCP aparece para o modelo como um conjunto adicional de ferramentas. [OpenCode +1](#)

Você poderia então pedir:

Analise a documentação arquitetural antes de implementar a autenticação.

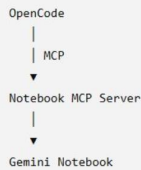
E o agente faria algo semelhante a:

```
knowledge.search_docs(
  "authentication architecture"
)
```

antes de modificar código.

3. Integração direta com Gemini Notebook

Em teoria:



com ferramentas:

```
list_notebooks()
get_sources()
ask_notebook()
add_source()
update_source()
```

Seria excelente.

Mas existe um problema:

precisamos de uma interface programática oficial e estável para o Gemini Notebook.

Hoje, a integração oficial descrita pelo Google está concentrada entre Gemini ↔ Gemini Notebook, e não em uma API pública genérica voltada a ferramentas externas. [Ajuda Google](#)

Portanto eu evitaria arquiteturas baseadas em:

browser automation
Playwright

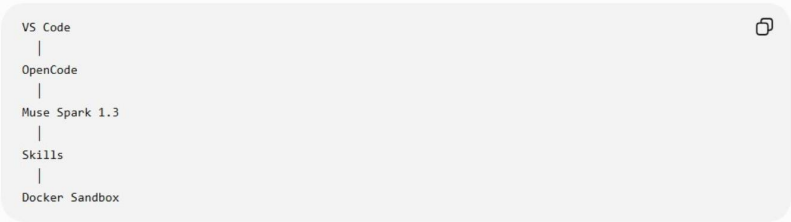
scraping
reverse-engineered API

Só faria isso experimentalmente.

É o tipo de solução que funciona hoje e quebra na próxima atualização do Google.

Onde eu vejo mais valor para você

Considerando a arquitetura que você está montando:



eu adicionaria uma camada:



Isso é melhor do que:

OpenCode → Notebook

porque transforma:

documentos

na fonte da verdade, e não:

Gemini Notebook

Existe ainda uma combinação mais interessante

Você pode usar o Gemini Notebook como **analista/revisor do projeto**, e não como memória operacional do agente.

Por exemplo:

OpenCode

```
implemente feature
↓
rode testes
↓
refatore
↓
produza CHANGELOG / ADR
```

Depois esses artefatos entram no Notebook:

```
ADR-001.md
ADR-002.md
architecture.md
requirements.md
test-results.md
```

E você usa o Gemini Notebook para perguntar:

"Quais decisões arquiteturais estão entrando em conflito?"

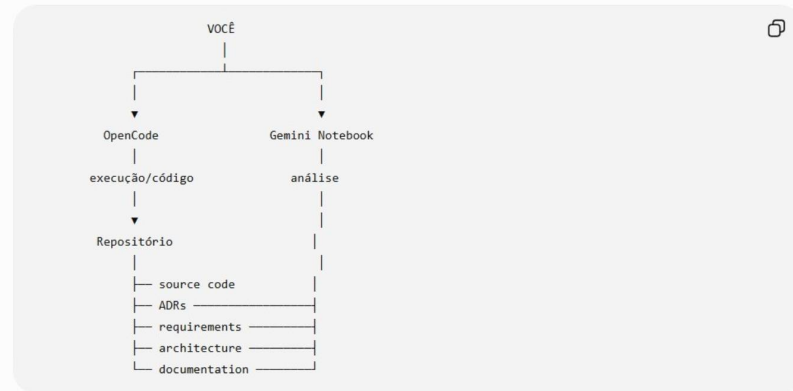
"Quais requisitos ainda não foram implementados?"

"Quais requisitos não foram implementados em ADR-001?"

"Quais decisões podem gerar dívida técnica?"

O Notebook é particularmente adequado a esse papel porque responde com base nas fontes selecionadas e fornece citações ligadas a elas. [Ajuda Google](#)

Um fluxo bastante poderoso seria



Nesse desenho:

OpenCode = executor

Gemini Notebook = analista / knowledge workspace

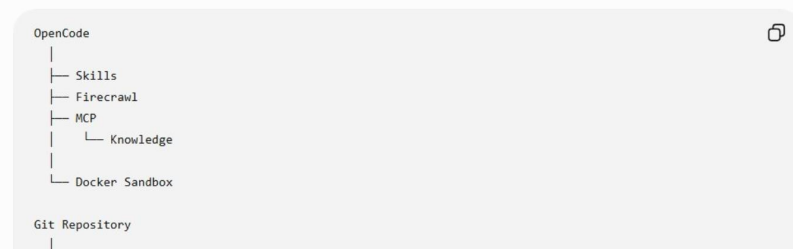
Git/docs = fonte da verdade

Esse desacoplamento é arquiteturalmente muito mais saudável.

Minha recomendação para o seu ambiente

Eu não gastaria esforço agora tentando integrar diretamente o OpenCode ao Gemini Notebook.

Eu montaria:



```
└─ docs/
   │ requirements
   │ architecture
   │ research
   │ ADR
   └─ references
Gemini Notebook
↑
└─ consume docs/references
```

E deixaria preparado o MCP para que, quando o Google disponibilizar uma interface pública apropriada para os Notebooks, você simplesmente substitua:

MCP → arquivos

por:

MCP → Gemini Notebook API

sem mudar o restante do ambiente.

Para a estrutura enxuta de **OpenCode + Skills + Docker Sandbox** que você está desenhando, esse é o desenho que considero com melhor relação entre **simplicidade, segurança, portabilidade e capacidade de evolução**.

   ...  Fontes

O ChatGPT pode cometer erros. Por isso, lembre-se de conferir informações relevantes.

+ Pergunte ao ChatGPT

Média ▾



Minha recomendação para o seu ambiente

Eu não gastaria esforço agora tentando integrar diretamente o OpenCode ao Gemini Notebook.

Eu montaria:

```
OpenCode
├─ Skills
├─ Firecrawl
├─ MCP
│ └─ Knowledge
└─ Docker Sandbox

Git Repository
├─
```